PES UNIVERSITY

SOFTWARE ENGINEERING MINI-PROJECT

SOFTWARE TEST PLAN

VEHICLE PARKING SYSTEM

Project Code: UE24CS341A

Prepared for: Software Engineering Mini-Project

Document Version: 1.0

Baseline: Team SRS v1.1 and Detailed Team Execution Handbook v1.1

Prepared by

Vehicle Parking System Project Team

12 October 2026

Technology: React + Vite | Django REST Framework | PostgreSQL target (SQLite local fallback if configured)

Status: Draft for team review

Repository: Vehicle-Parking-System-Team/Vehicle-Parking-System-SE-MINI-PROJECT

1. Introduction

Execution note: This plan defines what will be tested and how. Enter actual results, pass/fail status, coverage figures, CI status and tool configuration only after the corresponding checks have actually been run.

Purpose: This document defines the objectives, scope, approach, environment, responsibilities, schedule and completion criteria for testing the Vehicle Parking System. It guides the four-member team through unit, integration, system and user-acceptance testing while keeping tests aligned with the agreed SRS, software design and GitHub backlog.

Scope: Testing covers the application flow from authentication and vehicle registration through slot availability, reservations, check-in/check-out, billing, reporting, audit records, notifications and input validation. Both the React user interface and Django REST API are in scope, including persistence and role enforcement.

References: Team SRS v1.1 (FR-01–FR-30); Software Architecture/Design document (SAD); Vehicle Parking System Detailed Team Execution Handbook, baseline 1.1 dated 12 October 2026; GitHub user stories US-01–US-32; repository README/setup instructions; API contracts and data rules agreed by the team.

Definitions: FR = Functional Requirement; NFR = Non-Functional Requirement; SRS = Software Requirements Specification; SAD = Software Architecture/Design; RTM = Requirements Traceability Matrix; UAT = User Acceptance Testing; API = Application Programming Interface; CI = Continuous Integration.

Testing principles: The SRS is the source of truth. Do not add business rules that have not been agreed in the SRS/design. Mark unresolved rate, rounding, reservation-cutoff, response-time or other policy choices as TBD and resolve them with the team before asserting expected results.

2. Test Items

- Authentication and access management: login/logout, role-based permissions, account administration, profile and password flows.
- Vehicle management: register, view, update and search vehicles; show current parking status where available.
- Parking facility and availability: create/update/remove eligible slots; maintain available, occupied and unavailable states; show current status.
- Reservation management: create, modify and cancel future reservations; validate time windows, availability and overlapping/conflicting reservations.
- Parking operations: check-in, assign an eligible slot, prevent duplicate active assignments, check out, release the slot and retain parking history.
- Pricing, billing and payment records: configure rates, calculate fees using agreed rules, generate bill/receipt and record payment status without a real payment gateway.
- Admin dashboard and reporting: operational metrics, occupancy/usage reports, revenue summaries and permitted report exports.
- Audit and notifications: record significant actions and display relevant in-app notifications.
- Shared validation and security: required fields, malformed input, duplicate/conflicting state, safe error responses, authorization and security checks.

3. Features to be Tested

Requirement / Use Case	Feature coverage	Representative verification
FR-01–FR-02; FR-11; UC-02	Vehicle registration, view/update and search	Valid registration; missing fields; duplicate identifier; update/search existing and unknown vehicle; role restrictions.
FR-03–FR-04; FR-07; FR-10; UC-03	Slot lifecycle and current availability	Create/update eligible slot; reject unsafe removal; verify state transitions and current status consistency.
FR-05–FR-09; FR-12; UC-05–UC-06	Check-in, active assignment, checkout, slot release and history	Valid entry/exit; occupied/unavailable slot; duplicate assignment; failed checkout consistency; history retained.
FR-13–FR-14; UC-10	Common input validation and error handling	Missing/malformed/boundary data; duplicate/conflicting operations; consistent HTTP errors; no stack traces or secrets.
FR-15–FR-18; UC-01	Authentication, roles, account administration, profile/password	Valid/invalid login; logout invalidation; unauthorized/forbidden requests; profile/password validation.
FR-19–FR-22; UC-04	Reservation create, modify, cancel and conflict prevention	Valid future reservation; invalid time window; overlap; cutoff rules as specified; availability updated consistently.
FR-23–FR-26; UC-07	Rates, fee calculation, bills/receipts and payment status	Valid/invalid rate; fee boundary/rounding based on agreed rules; bill reconciliation; valid/invalid payment transition.
FR-27–FR-28; UC-08	Dashboard, occupancy/usage/revenue reports and export	Metrics reconcile with source records; date filters; empty data; access control; exported file content/format.
FR-29–FR-30; UC-09	Audit log and in-app notifications	Audit actor/time/action/target; secrets excluded; notification creation/read state; access restrictions.
NFR-SEC-01–07	Security hardening and verification	Authorization bypass attempts, input abuse, secret exposure, unsafe error messages and static/security findings.

4. Features Not to be Tested

- Real-money payment processing and third-party payment gateway behavior; agreed scope is to record payment status without requiring a real gateway.
- Production deployment infrastructure, campus gate hardware, physical barriers, cameras, RFID readers or other devices not included in the agreed application scope.
- External identity providers or services not specified in the SRS and unavailable in the test environment.
- Database engine internals and framework internals beyond the application's use of them; application-level data integrity, queries and persistence remain in scope.
- Unspecified business policies such as rate unit, rounding, minimum charge, overnight charging or reservation cancellation cutoff are not fixed expected behavior until the team confirms them in the SRS/design.

5. Test Approach / Strategy

5.1 Test levels

Level	Purpose	Examples / owner
Unit testing	Verify individual functions, validators, permission rules and fee/status calculations in isolation.	Django/Python unit tests; frontend component tests where configured. Story owners maintain tests for their code.
Integration testing	Verify interactions across frontend/API, API/database and module boundaries.	Vehicle-to-check-in; slot availability-to-reservation; checkout-to-billing; billing-to-reports; audit/notification events.
System testing	Verify end-to-end workflows using the integrated application.	Login → register vehicle → check in/assign slot → checkout → bill/receipt → view history/report.
Acceptance testing (UAT)	Confirm user-visible behavior meets acceptance criteria.	Administrator, Operator and Authorized User scenarios using agreed role permissions and acceptance criteria.

5.2 Test types

- Functional testing: validate every in-scope requirement and each use case.
- Negative and boundary testing: missing fields, malformed input, unknown identifiers, duplicate data, invalid state transitions, date/time boundaries and conflicting requests.
- Regression testing: rerun relevant tests after fixes, merges or changes to shared contracts.
- API testing: validate request/response schemas, HTTP status codes, permissions, validation messages and persistence.
- UI/usability testing: check navigation, clear status labels, readable validation messages, empty states and prevention of accidental destructive actions.
- Security testing: verify backend authorization for each role; hiding frontend controls alone is not sufficient; ensure errors and audit data do not expose secrets.
- Performance/concurrency testing: observe response and behavior under representative normal and simultaneous requests. Numeric targets must come from the agreed NFR/SRS; none are invented here.
- Data integrity testing: successful checkout must close the active session, release its slot and preserve history consistently; failed operations must not leave partial updates.

5.3 Test design and execution rules

- Use equivalence partitions, boundary-value analysis, decision tables and state-transition testing where appropriate.
- Each implemented use case must have at least 8–10 test cases across relevant unit, integration and system levels, including valid and invalid cases, as required by the team handbook.
- Each test case records a stable ID, linked FR/NFR and user story, preconditions, test data, steps, expected result, actual result, status, defect link and execution date.
- Keep the actual result and Pass/Fail as Not Run until a tester executes the case. Do not report planned checks as completed tests.
- Link each story and test evidence through GitHub issues/PRs; review changes and run applicable checks before merging.

Entry criteria: a reviewable build/branch is available; required environment and dummy data are ready; acceptance criteria and API/data contracts are agreed; blocking dependencies are identified; and test cases are reviewed.

Exit criteria: all planned cases for the release scope are executed or explicitly deferred with approval; every implemented use case has the required 8–10 or more cases; critical/high-priority defects are fixed or formally accepted; no unresolved release-blocking defect remains; requirement coverage and regression results are documented; and the team records acceptance. Unmet conditions must be reported.

5.4 Security Validation

- Verify login/logout and role-based authorization at the backend API, including direct requests that bypass the UI.
- Check that passwords, tokens, secrets and sensitive values are not written to logs, API errors, audit records or committed configuration.
- Check input validation and safe handling of malformed or unexpectedly large values; errors must not expose stack traces or internal implementation details.
- Check access restrictions for account administration, slot operations, reservations, billing records, reports, audit logs and notifications.
- Review dependencies/configuration and run security/static-analysis tools only if actually configured. Record tool, version, workflow/command, result and unresolved findings; do not claim Jenkins or SonarQube is configured/passing without evidence.

- Security acceptance criteria are NFR-SEC-01–07 from the agreed SRS. Exact wording and numeric thresholds must come from that SRS.

6. Test Environment

Area	Planned setup / control
Frontend	React + Vite application; use repository setup instructions. Verify `npm run build` when frontend changes are included.
Backend/API	Django + Django REST Framework; run Django checks and relevant automated tests using the documented environment.
Database	PostgreSQL is the target. SQLite may be the current local fallback until PostgreSQL integration is configured. Record the database used for each test run and repeat data-integrity tests on the target database before sign-off.
CI	GitHub Actions exists for backend checks and frontend build. Inspect and record the actual latest run; do not assume it passes. Jenkins is a later deliverable and must not be reported as configured unless verified.
Browsers / devices	Use the team's available desktop browser(s). Record browser/version for reproducible UI defects and broaden coverage according to team/course requirements.
Tools	Django test framework/unittest or pytest if already configured; browser developer tools; API client such as Postman/curl if available; GitHub Issues/Projects/PRs. Use JMeter or other load tooling only if available and needed.
Test data	Synthetic users for Administrator, Operator and Authorized User; unique/duplicate vehicle identifiers; slot states; future/conflicting reservations; active/completed parking sessions; test rates; bill/payment records; empty and populated reports.
Safety	Use dummy data only. Never place real passwords, access tokens, personal data or payment details in committed fixtures, screenshots or logs.

7. Test Schedule

Dates below use the professor's supplied checkpoint and sprint plan. Detailed test execution dates after Sprint 2 are to be confirmed with the team; this plan does not imply testing has already occurred.

Milestone	Planned date / window	Expected evidence
Backlog and test-plan checkpoint	12 October 2026	Test plan draft, requirement/story links, known TBDs and ownership reviewed.
Practice / dry-run window	12–16 October 2026	Environment/setup rehearsal, smoke-test checklist and early defect logging where a runnable build exists.
Sprint 1	19–23 October 2026	Unit tests and focused integration tests for stories actually implemented; evidence linked to PRs.
Sprint 2	26–30 October 2026	Tests for Sprint 2 stories, regression of affected Sprint 1 behavior and integrated workflow checks where dependencies are ready.
Integrated system regression and UAT	To be agreed after Sprint 2	End-to-end results, defect retests, acceptance review and final summary.
Release / final sign-off	To be agreed	Approved test summary, known limitations and explicit acceptance decision.

8. Test Deliverables

- This Software Test Plan and reviewed revisions.
- Test case suite covering each implemented use case (minimum 8–10 cases per use case), with requirement and user-story links.
- Reusable test data and setup instructions that contain no secrets.
- Unit/integration/system test code and API/UI test scripts where practical.
- Execution log containing tester, date/build, environment/database, actual result and Pass/Fail/Blocked/Not Run status.
- Defect reports linked to GitHub issues, including severity/priority, reproduction steps, expected vs actual result and evidence.

- Requirements Traceability Matrix and coverage summary.
- CI run links/logs, security/static-analysis evidence if actually available, regression results and UAT feedback.
- Final Test Summary Report recording scope executed, results, open defects, limitations, deviations and acceptance decision.

9. Roles and Responsibilities

Role	Assigned responsibility	Accountability
Member 1 — Identity, access and vehicle management	Own tests for US-01–US-08; contribute to shared validation and master traceability.	Authentication/role/vehicle flows, requirement links and integration coordination.
Member 2 — Parking facility and reservations	Own tests for US-09–US-16.	Slot lifecycle/status, reservation availability and conflict rules.
Member 3 — Parking operations and billing; Test Plan lead	Own tests for US-17–US-24; coordinate test strategy, unit/integration testing and coverage evidence.	Check-in/out integrity, billing/payment-record behavior, test-plan maintenance and consolidated execution reporting.
Member 4 — Dashboard, audit, notifications and quality	Own tests for US-25–US-32; coordinate validation/security evidence with all members.	Reports, audit/notifications, common error contract and security validation.
All developers / story owners	Implement tests alongside each vertical slice; reproduce/fix defects; rerun relevant tests; attach evidence to PRs.	No test is marked passed without execution evidence.
Team / Product Owner or instructor-designated approver	Review acceptance criteria, clarify TBD business rules and approve release readiness.	Review test summary, known limitations and unresolved defects.

10. Risks and Mitigation

Risk	Potential effect	Mitigation / response
Unstable or incomplete build	Tests cannot run or results are misleading.	Use smoke tests on each available build; mark blocked/not run; coordinate dependencies before system testing.
Unclear/changing business rules	Expected results differ between modules.	Resolve rate/rounding, reservation cutoff, overlap, role and deletion rules in SRS/API contract; record decisions and update tests.
Local database differs from target	Persistence or transaction behavior may differ.	Record database engine for every run; test core data-integrity flows on PostgreSQL before sign-off.
Module/API contract mismatch	Integrated workflows fail despite isolated tests passing.	Agree request/response and error contracts; add integration tests and review linked PRs.
Concurrent requests cause double booking	Two users claim one slot or vehicle state becomes inconsistent.	Test simultaneous conflicting operations and verify backend/database rejects conflicts and preserves one valid state.
Insufficient test data or role accounts	Coverage gaps and blocked cases.	Maintain resettable synthetic fixtures for all roles, slot states, reservation conflicts and billing/report cases.
Security or sensitive-data exposure	Unauthorized access or leakage in logs/errors.	Test backend permissions, safe errors and secret exclusion; record and resolve findings.
False completion claims or missing evidence	Project sign-off is not defensible.	Keep status Not Run until executed; link evidence, CI run and defects; report incomplete work honestly.

11. Assumptions & Dependencies

- The team's SRS v1.1 (FR-01–FR-30) and agreed security requirements NFR-SEC-01–07 are the baseline. If the submitted SRS differs from the handbook, reconcile it explicitly before changing this plan.
- The React frontend, Django/DRF backend and a usable database/test configuration will be available as development progresses; current readiness must be checked rather than assumed.
- PostgreSQL is the target database, while SQLite may be used temporarily for local development. Database-specific evidence must identify the actual engine.
- Dummy test users and records can be created and reset safely.
- Module owners will provide or agree API contracts for vehicles, slots, reservations, parking sessions, billing, reports, audit events and notifications.
- Business rules not explicitly defined in source documents (fee rounding/rate unit, reservation cutoff and destructive-operation rules) remain TBD until approved.

- GitHub Actions exists, but the latest result must be inspected. Jenkins, SonarQube and other optional tooling are not assumed configured or successful.
- Final integrated testing and UAT dates depend on implementation readiness and will be agreed by the team.

12. Suspension & Resumption Criteria

Suspend the affected test suite when:

- The application or test environment is unavailable and prevents meaningful execution.
- A blocking defect prevents a major workflow from running or makes expected results unreliable.
- More than 30% of the cases planned for the current test cycle are blocked by the same build/environment issue.
- Test data is corrupted, the database is in an unknown state, or a security concern makes continued testing unsafe.

Resume when:

- The blocking defect is fixed or an approved workaround is documented and verified.
- The environment and database are stable, test data has been reset/validated, and the build/version is recorded.
- Previously blocked smoke tests pass and the test lead confirms the affected suite can continue.

Record suspension date, build, reason, affected cases and owner. Resume from the last known valid checkpoint and rerun tests affected by the fix.

13. Test Case Management & Traceability

Each test case receives a stable ID (for example, TC-UC05-INT-001) and is linked to the relevant SRS requirement, GitHub user story, test level, code/PR where applicable and defect. Store the suite in the repository's testing documentation area when confirmed; the handbook lists `docs/testing/` as expected, but inspect the actual repository before adding or replacing files.

Use case / requirement group	Story mapping	Minimum coverage focus	Example test IDs
UC-01: FR-15–FR-18	US-01–US-04	8–10+ cases; success, invalid credentials, access denial, logout/session, profile/password.	TC-UC01-UNIT-001; TC-UC01-SYS-001
UC-02: FR-01, FR-02, FR-11	US-05–US-08	Registration, required/duplicate fields, update, search and permissions.	TC-UC02-UNIT-001; TC-UC02-API-001
UC-03: FR-03, FR-04, FR-10; related FR-07	US-09–US-12	Slot lifecycle, state transitions, status consistency, unsafe deletion/assignment.	TC-UC03-UNIT-001; TC-UC03-INT-001
UC-04: FR-19–FR-22	US-13–US-16	Valid future reservation, invalid window, modification/cancellation, overlap and availability.	TC-UC04-API-001; TC-UC04-SYS-001
UC-05: FR-05–FR-07	US-17–US-18	Check-in, no available slot, unknown vehicle, occupied/unavailable slot, duplicate assignment, concurrency.	TC-UC05-UNIT-001; TC-UC05-INT-001
UC-06: FR-08, FR-09, FR-12	US-19–US-20	Checkout, release slot, history retention, repeat checkout and atomic consistency.	TC-UC06-INT-001; TC-UC06-SYS-001
UC-07: FR-23–FR-26	US-21–US-24	Rate validation, fee calculation using agreed rules, bill/receipt and payment status transition.	TC-UC07-UNIT-001; TC-UC07-SYS-001
UC-08: FR-27–FR-28	US-25–US-28	Dashboard metrics, empty dataset, date filter, occupancy/revenue reconciliation, role access and export.	TC-UC08-API-001; TC-UC08-SYS-001
UC-09: FR-29–FR-30	US-29–US-30	Audit actor/time/action/target, no secrets, notification creation/read state and access restrictions.	TC-UC09-INT-001; TC-UC09-SYS-001
UC-10: FR-13–FR-14; NFR-SEC-01–07	US-08, US-31–US-32 and all modules	Malformed/boundary/duplicate input, safe errors, authorization and security checks.	TC-UC10-API-001; TC-SEC-001

13.1 Representative test scenarios

These are design examples, not execution results. Expand each implemented use case to at least 8–10 test cases across relevant levels. Actual outcomes must be recorded only after execution.

Test ID	Level	Scenario / precondition	Expected result	Initial status
TC-UC01-SYS-001	System	Submit valid credentials for an active test user.	User is authenticated and routed to an allowed view.	Not Run
TC-UC01-API-002	Integration	Call a protected endpoint without authentication.	Request is rejected with agreed status; no protected data is returned.	Not Run
TC-UC02-API-001	Integration	Register a vehicle with all required valid fields.	Vehicle is saved and can be retrieved with the same identifier.	Not Run
TC-UC02-API-002	Integration	Register a vehicle using an identifier that already exists.	Duplicate is rejected clearly; no second vehicle record is created.	Not Run
TC-UC03-UNIT-001	Unit	Attempt to remove an occupied slot or one with an active assignment.	Operation is rejected according to the agreed integrity rule; existing state is preserved.	Not Run
TC-UC03-SYS-002	System	View status after a slot changes from available to occupied.	UI state is consistent with the backend record.	Not Run
TC-UC04-API-001	Integration	Create a future reservation for a valid available time window.	Reservation is stored and availability reflects it according to the agreed rule.	Not Run
TC-UC04-API-002	Integration	Create an overlapping reservation for the same incompatible slot/time.	Conflict is rejected; existing reservation remains unchanged.	Not Run
TC-UC05-INT-001	Integration	Operator checks in a registered vehicle and selects an available slot.	One active session is created and the slot becomes occupied.	Not Run
TC-UC05-INT-002	Integration	Attempt check-in for a vehicle that already has an active assignment.	Second assignment is rejected; no duplicate session/slot allocation occurs.	Not Run
TC-UC06-INT-001	Integration	Check out a vehicle with an active session.	Session closes, slot becomes available and completed history remains accessible.	Not Run
TC-UC06-INT-002	Integration	Simulate checkout failure at a safe test boundary.	No partial state remains; session and slot state remain consistent.	Not Run
TC-UC07-UNIT-001	Unit	Calculate a fee using an agreed rate and session duration.	Fee matches documented rate and rounding rules.	Not Run
TC-UC07-API-002	Integration	Attempt an invalid payment-status transition.	Transition is rejected and prior status remains unchanged.	Not Run
TC-UC08-SYS-001	System	Generate an occupancy report for a known test period.	Totals reconcile with underlying slot/session test data.	Not Run
TC-UC08-API-002	Integration	Request a restricted report as a role not permitted to view it.	Access is denied and restricted data is not returned.	Not Run
TC-UC09-INT-001	Integration	Perform a significant action such as a slot update.	Audit record captures actor, timestamp, action and target without secrets.	Not Run
TC-UC09-SYS-002	System	Trigger an in-scope event that should create a notification.	Intended user sees notification; read/unread behavior matches specification.	Not Run
TC-UC10-API-001	Integration	Submit missing, malformed or out-of-range input to an API endpoint.	Request is rejected consistently with safe actionable error; no stack trace is exposed.	Not Run
TC-SEC-001	Security	Attempt an administrator-only API operation as an unauthorized role.	Backend denies the request even if the UI is bypassed.	Not Run
TC-SEC-002	Security	Inspect logs, audit data and error responses after invalid requests.	No password, token, secret or sensitive configuration value is exposed.	Not Run

13.2 Execution record template

Field	Value to record
Test ID / linked FR / US	Stable ID and requirement/story references
Build / commit / environment	Commit SHA or build identifier; browser; database engine; test configuration
Tester / execution date	Name/team role and date actually executed
Preconditions / test data	Synthetic records and starting state
Actual result / status	Observed behavior; Pass / Fail / Blocked / Not Run
Defect / evidence	GitHub issue or PR link, logs/screenshots with secrets removed
Retest / closure	Fix version and retest result, when applicable

14. Test Metrics & Reporting

- Execution progress = executed cases ÷ planned cases × 100%. Report blocked and not-run cases separately.
- Pass rate = passed cases ÷ executed cases × 100%. Do not include Not Run or Blocked in the denominator.
- Requirement coverage = requirements with at least one executed linked test ÷ in-scope requirements × 100%.
- Defect counts by severity/priority, open/closed state, age and affected requirement/use case.
- Regression outcome and retest success for fixed defects.
- Coverage/tool results only when the relevant tool has actually run; identify the tool, command/workflow and report artifact.
- Daily or per-session execution update during active testing; consolidated Test Summary Report after the agreed test cycle.

Reporting rule: each report identifies the build/commit, environment/database, date, tester, test counts, defects, known limitations and any untested requirement. A successful build/check is not a substitute for functional test evidence.

15. Approvals

Approval confirms that the plan is understood and unresolved requirements are tracked. It does not claim the product has passed testing.

Role	Name	Signature / Date
Test Plan Lead — Member 3		
Team / Development Lead		
Product Owner / Instructor-designated approver		

Appendix A. Requirement-to-test ownership overview

Epic / module	Stories	Primary owner	Integration dependencies
E1 Identity & Access Management	US-01–US-04	Member 1	Role/session behavior used by every protected module.
E2 Vehicle Management	US-05–US-08	Member 1	Vehicle records consumed by reservations and parking operations.
E3 Parking Facility & Availability	US-09–US-12	Member 2	Slot state/integrity consumed by reservations and check-in/out.
E4 Reservation Management	US-13–US-16	Member 2	Reservation/availability data shared with check-in operations.
E5 Parking Operations	US-17–US-20	Member 3	Consumes vehicle and slot contracts; publishes active/completed parking sessions.
E6 Pricing, Billing & Payment	US-21–US-24	Member 3	Consumes completed sessions; publishes billing/payment records to reports.
E7 Admin Dashboard & Reporting	US-25–US-28	Member 4	Consumes parking, reservation, billing and payment data.

E8 Audit, Notifications & Quality	US-29–US-32	Member 4	Shared event/error contracts and security validation across modules.

Appendix B. Defect priority guide

Priority	Definition	Expected handling
P0 — Critical	Core data corruption, unauthorized sensitive access, or failure making the primary system unusable.	Stop affected testing; notify the team immediately; do not sign off while unresolved.
P1 — High	Major workflow such as check-in/out, slot allocation, reservation conflict handling or billing is incorrect with no acceptable workaround.	Fix and retest before acceptance unless formally waived by the approver.
P2 — Medium	Feature is impaired but a reasonable workaround exists; limited incorrect behavior or reporting issue.	Prioritize within the sprint/release and document the decision.
P3 — Low	Minor usability, wording or visual issue without material functional/data impact.	Record and schedule based on team priority.